Abstraction & Polymorphism

* Abstraction
→ the power of focusing on what something does, while ignoring how it does that

Human ⟶ Engineer
Engineer ⟶ Physicist
Physicist ⟶ Mathematician

eg: ATM Machine
- deposit
- withdraw
- check Balance

Behaviours

Methods

OOP ⟶ (x) Real Word (x)
↳ Humans think about Reality

• Abstraction (cont)
↳ Represent whatever necessary
↳ Even what we model, we don't want everyone to know how everything works, but they can still use it

Java
↳ low level Abstraction (Hiding implementation Details)
↳ High level Abstraction (Separate WHAT form How)

Page No.:
Date:
YOUVA SELLAR

Low level Abstraction

```
class Car {
    String type;

    start() {
    }

    accelerate() {
    }

    brake() {
    }
}
```

→ Concrete Classes

```
main() {
    Car c = new Car();
    c.start();
    c.accelerate();
    c.brake();
}
```

High level Abstraction: (Seperate WHAT from HOW)

- abstract classes
- Interfaces

```
class FuelCar {
    drive() {
        // code //(1)
    }
}
```

```
class ElectricCar {
    drive() {  //(2)
        //code
    }
}
```

```
main() {
    ElectricCar ec = new ElectricCar();
    FuelCar fc = new FuelCar();
    ec.drive();
    fc.drive();
}
```

```
class Car {
    applyHandBrake() {
        // code
    }
}
```

```
class FC extends Car {
    drive() { }
}
```

```
class EC extends Car {
    drive() { }
}
```

```
Car c = new EC();
        FC();
```

* Abstract Class

```
abstract class Car {
    start() { }

    abstract accelerate();
    abstract brake();
}
```

```
class EC extends Car {
    accelerate() { // code }
    brake() { }
}
```

```
class FuelCar extends Car {
    accelerate() {
        // code
    }
    brake() { }
}
```

* Interface
  - abstract class
  - Contract

class ⟶ Blueprint of Object
Interface ✗ ⟶ Object

```
interface Car {
    void start();
    void accelerate();
    void brake();
}
```

implement keyword used to use interface

```
class EC implements Car {
    ____
    ____
    ____
}
```

Car c = new ____

```
class FC implements Car {
    ____ startA() {}
    ____ acc() {}
    ____ break() {}
}
```

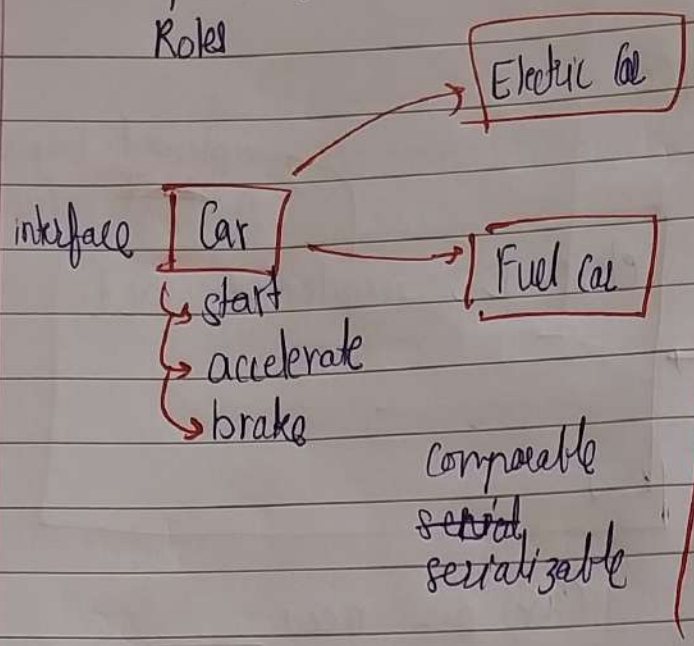
Interface
↳ Pure WHAT?
(Pure Abstraction)

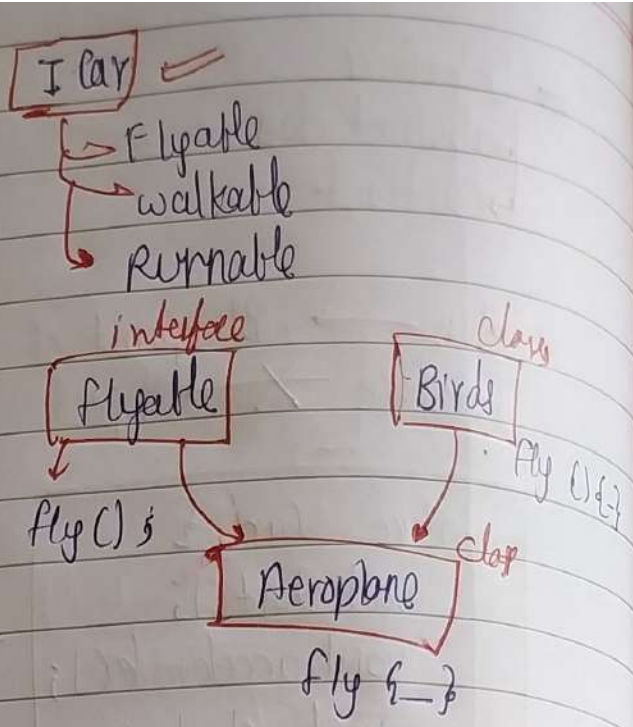
Contract

Abstract Class
partial WHAT & HOW?

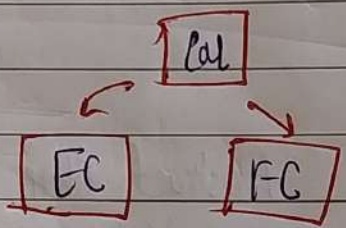
Intention

Interfaces
↳ responsibilities / capabilities / Roles





Abstract class
↳ families of similar Object



* Multiple Inheritance
↳ class ✗
↳ inheritance (✓)

Abstraction
↓
data / implementation Hiding

Encapsulation
① { = }
② Data-security
↳ Access modifiers

Date:

* Polymorphism

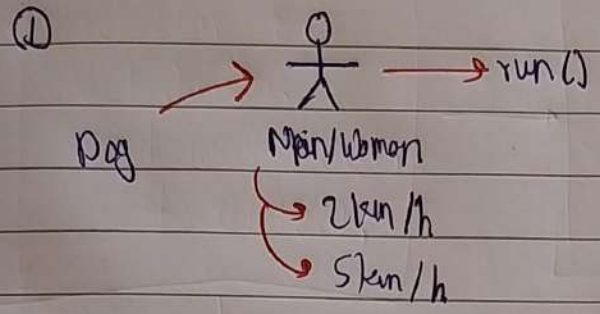
many — form → Real life objects

They have many forms

Polymorphism

Static polymorphism

①



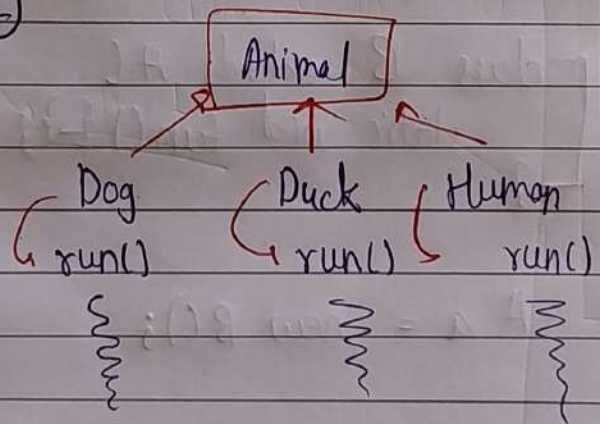
Some obj. behaves/responds differently on same command if one param is changed (compile-time polymorphism)

Method overloading

```
class Human {
  void run() {
    -2km/h
  }
  void run (boolean isDogBehind)
  {
    if (true) {
      -5km/hr
    }
  }
}
```

Dynamic polymorphism

②




```
Animal a = new Dog();
a.run();
```

Run time polymorphism

Method overriding

```
abstract class Animal {
  abstract void run();
}

class Dog extents Animal
  run() { ___ }
}

class Duck " " {
  " " }
```

- Same for Human class

```
Animal a;
if (type == "duck") {
    a = new Duck();
}
else if(____) {
    a = new Human();
}
```

Polymorphism

compile time (method overloading)	Run time (Method overriding)

static, final, Private

static
↳ override ✗

```
class A {
    static void fun() {___};
}
```

```
class B extends A {
    static void fun() {};
}
```

```
A a = new B();
```